

Gmail MCP Troubleshooting Guide

Connecting an n8n Gmail workflow to Claude Desktop through supergateway — the working configuration for every platform, and a fix for each way it breaks.

Covers · macOS Apple Silicon · macOS Intel · Windows 10 / 11

— Contents

- 1 How the pieces fit together
 - 2 The config file — where it lives, what goes in it
 - 3 The n8n side — trigger node and tool nodes
 - 4 Verify in three steps
 - 5 Universal problems (any platform)
 - 6 macOS — Apple Silicon
 - 7 macOS — Intel
 - 8 Windows 10 / 11
 - 9 Reading the logs
 - 10 Error → fix quick reference
 - 11 Security before you go live
-

Who this is for

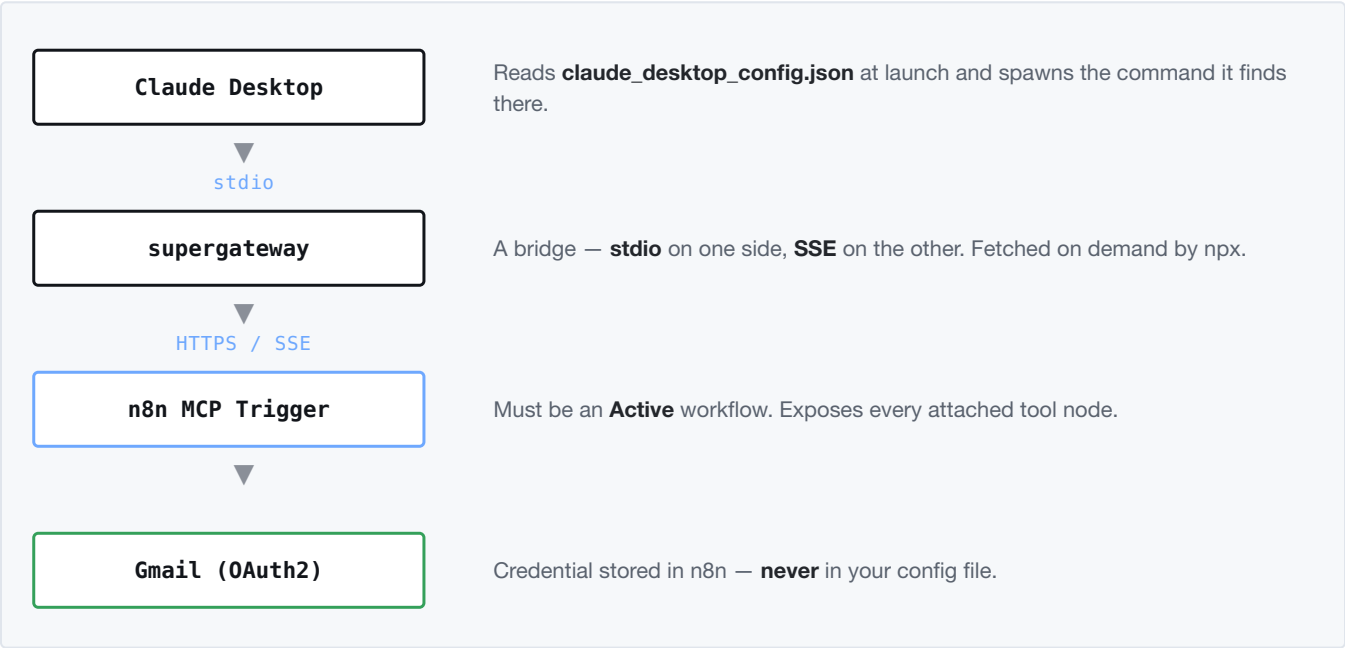
Anyone wiring an n8n Gmail workflow into Claude Desktop as an MCP server. The setup is four moving parts, and a failure in any one of them surfaces as the same unhelpful symptom: the server shows red, or simply never appears. This guide separates them so you can tell which part is actually broken.

THE 80% ANSWER

Two mistakes cause most failures: using a bare `npm` instead of its absolute path, and copying the **Test** URL from n8n instead of the **Production** URL. Check those two before reading anything else.

1 How the pieces fit together

Four components in a chain. Each one can fail independently, and each fails with a different signature.



If this breaks...	You see	Jump to
Config file	Server missing from Claude entirely, or JSON error on launch	Section 2, 5
npx / node path	Server appears but red; <code>ENOENT</code> in the log	Sections 6, 7, 8
supergateway ↔ n8n	Server red; SSE or 404 errors in the log	Section 5
n8n workflow	Server green, tools listed, but calls fail	Sections 3, 5
Gmail credential	Tools run, then return a 403	Section 5

WHY SUPERGATEWAY IS NEEDED AT ALL

Claude Desktop talks to MCP servers over **stdio** — it launches a process and pipes to it. n8n publishes its MCP endpoint over **SSE** (HTTP). supergateway is the adapter between the two. It is not something you install permanently; `npx -y` fetches and runs it each time Claude starts.

2 The config file

Where it lives

Platform	Path
macOS (both chips)	<code>~/Library/Application Support/Claude/claude_desktop_config.json</code>
Windows	<code>%APPDATA%\Claude\claude_desktop_config.json</code>

If the file does not exist, create it. On macOS the folder is hidden in Finder — press `⇧⌘G` and paste the path.

macOS — Apple Silicon (M1 / M2 / M3 / M4) MOST COMMON

```
{
  "mcpServers": {
    "gmail-assistant": {
      "command": "/opt/homebrew/bin/npx",
      "args": [
        "-y",
        "supergateway",
        "--sse",
        "https://YOUR-INSTANCE.app.n8n.cloud/mcp/YOUR-PATH"
      ],
      "env": {
        "PATH": "/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin"
      }
    }
  }
}
```

macOS — Intel

Identical, but every `/opt/homebrew` becomes `/usr/local`:

```
{
  "mcpServers": {
    "gmail-assistant": {
      "command": "/usr/local/bin/npx",
      "args": [
        "-y",
        "supergateway",
        "--sse",
        "https://YOUR-INSTANCE.app.n8n.cloud/mcp/YOUR-PATH"
      ],
      "env": {
        "PATH": "/usr/local/bin:/usr/bin:/bin"
      }
    }
  }
}
```

Windows 10 / 11

Windows needs `cmd /c` in front of `npx`, and no `env` block:

```
{
  "mcpServers": {
    "gmail-assistant": {
      "command": "cmd",
      "args": [
        "/c",
        "npx",
        "-y",
        "supergateway",
        "--sse",
        "https://YOUR-INSTANCE.app.n8n.cloud/mcp/YOUR-PATH"
      ]
    }
  }
}
```

MERGING INTO AN EXISTING FILE

If you already have MCP servers configured, paste **only** the `"gmail-assistant": { ... }` object inside your existing `mcpServers`, with a comma after the previous entry. Two `mcpServers` keys in one file is not a merge — the last one silently wins and every server in the first block disappears. This is one of the most common and most confusing failures.

Two values you must replace

1. **The URL.** From your n8n MCP Server Trigger node, copy the **Production URL** (`/mcp/...`). Not the Test URL.
2. **The npx path (macOS only).** If node came from nvm, Volta, fnm or asdf, it is not in Homebrew. Run `which npx` and use whatever it prints.

3 The n8n side

One trigger node, plus one tool node per capability you want to expose.

Node 1 — MCP Server Trigger

Setting	Value
Node type	n8n-nodes-langchain.mcpTrigger
Path	gmail-assistant — set this explicitly rather than keeping the auto-generated UUID
Authentication	None to start. See Section 11 before real use.
URL to copy	The Production URL , ending /mcp/gmail-assistant

Nodes 2–4 — Gmail tool nodes

Drag each onto the trigger's **Tool** connector — the small diamond underneath the node, not the main output arrow. All three share one Gmail OAuth2 credential.

Node name	Resource	Operation
Get many messages in Gmail	Message	Get Many
Create a draft in Gmail	Draft	Create
Send a message in Gmail	Message	Send

THE NODE NAME BECOMES THE TOOL NAME

Claude sees these nodes as `Get_many_messages_in_Gmail` and so on. Renaming a node silently renames the tool for every client already pointed at it. Give them descriptive names once and leave them alone.

Leave the parameters AI-fillable

On each tool node, the fields Claude needs to control — `To`, `Subject`, `Message`, `Search` — must be switched to **"Let the model fill this in"** (the small star/AI toggle beside each field). Hardcoding them makes the tool inert: Claude can call it but cannot steer it.

THE ERROR THAT STOPS ALMOST EVERYONE

n8n marks every AI-fillable field as **required**, even ones that look optional. Call a tool without all of them and you get:

```
MCP error -32602: Invalid arguments ... expected string, received undefined
```

This is **not** a broken connection. It means the model omitted a field n8n insists on — typically `Received_After` and `Received_Before` on the Get Many node. Either supply every parameter on each call, or remove the fields you do not want from the node so n8n stops demanding them.

Activate the workflow

Toggle the workflow **Active** in the top-right of the editor. The Production URL returns 404 until you do. The Test URL only answers while you are sitting in the editor with "Listen for test event" clicked, and dies the moment you navigate away — which is why setups that "worked yesterday" are dead today.

4 Verify in three steps

Test the chain from the far end backwards. This tells you which component is broken instead of guessing.

Step 1 — Is the n8n endpoint alive?

Run this in Terminal (macOS) or Command Prompt (Windows), with your own URL:

```
npx -y supergateway --sse https://YOUR-INSTANCE.app.n8n.cloud/mcp/YOUR-PATH
```

Expected: a banner listing `outputTransport: stdio`, then `Stdio server listening`, then it hangs open. That is success — press `Ctrl+C` to exit.

If it exits immediately the problem is n8n, not Claude: workflow inactive, wrong URL, or you copied the Test URL.

Step 2 — Is the JSON valid?

```
# macOS
python3 -m json.tool ~/Library/Application\ Support/Claude/claude_desktop_config.json

# Windows
python -m json.tool %APPDATA%\Claude\claude_desktop_config.json
```

It prints your config back if valid, or names the line if not. A trailing comma after the last entry is the usual culprit.

Step 3 — Restart Claude properly

Closing the window is not a restart. Claude keeps running.

- **macOS:** `⌘Q`, or right-click the Dock icon → Quit. Confirm it is gone from the Dock before reopening.
- **Windows:** close the window, then find Claude in the system tray (bottom-right, possibly under the `^` arrow), right-click → Quit. If unsure, open Task Manager and end every `Claude.exe` process.

SUCCESS LOOKS LIKE

Claude Desktop shows the server connected, and the Gmail tools become available. In the log you will see `Server started and connected successfully` followed by `[supergateway] Stdio server listening`.

5 Universal problems ANY PLATFORM

Server never appears in Claude at all

Claude did not read your file. In order of likelihood: invalid JSON (run Step 2); the file is in the wrong folder; you edited a `.txt` copy because the OS hid the real extension; you have two `mcpServers` keys and yours is in the losing one.

Server appears but shows as failed / red

The file was read and the command was launched, but the process died. This is a path problem on macOS (Sections 6–7) or a `cmd /c` problem on Windows (Section 8). Read the log — it names the exact binary it tried.

404 from n8n

- Workflow is not **Active**. Most common cause by a wide margin.
- You used the Test URL (`/mcp-test/`) instead of Production (`/mcp/`).
- The path in the trigger node was changed after you copied the URL.

"SSE error: undefined" repeating in the log USUALLY HARMLESS

DO NOT CHASE THIS ONE

n8n Cloud closes idle SSE connections; supergateway notices and reconnects. On a healthy, fully working install this error can appear dozens of times over a few hours while every tool call succeeds normally.

Only investigate if tools are actually failing. If your tools work, ignore it entirely.

Tools appear but return 403 when called

The Gmail OAuth2 credential lacks the scopes for the operation. A read-only credential connects fine and lists tools fine, then fails the moment you send or draft. Re-authorize the credential in n8n from scratch rather than editing it — editing does not re-prompt for scopes.

It worked, then broke after a few days

`npx -y supergateway` pulls the latest version every launch, so a bad release can break a working setup with no change on your side. Pin it:

```
"supergateway@3.4.0" // instead of "supergateway"
```

Check the current version with `npm view supergateway version` and pin to one you have seen working.

Corporate network / VPN

Two separate things can be blocked: the npm registry (npx cannot download supergateway) and the long-lived SSE connection to n8n (proxies often kill streaming responses). If Step 1 fails on a work network but succeeds on a phone hotspot, this is your answer.

Why the absolute path is mandatory

Claude Desktop is a GUI app. macOS launches GUI apps with a minimal environment that does **not** include `/opt/homebrew/bin` — your shell's `PATH` comes from `.zshrc`, which only runs for terminal sessions. So `npx` works perfectly when you test it in Terminal and fails when Claude tries the identical command.

This is the single largest source of failures, and it looks like the config is being ignored when it is not.

Symptom

```
spawn npx ENOENT
```

Fix

Use the absolute path as `command`, and keep the `env.PATH` block so `npx` can find `node` after it starts. Both are needed.

Finding your actual npx

```
which npx
```

If it prints...	Use that exact path
<code>/opt/homebrew/bin/npx</code>	Standard Homebrew. Use the Section 2 config as-is.
<code>~/.nvm/versions/node/...</code>	nvm. Use the full printed path; set <code>PATH</code> to that directory plus <code>:/usr/bin:/bin</code> .
<code>~/.volta/bin/npx</code>	Volta. Same approach.
<code>~/.local/bin/npx</code>	A shim. Prefer the real Homebrew path — run <code>which -a npx</code> to see all of them and pick <code>/opt/homebrew/bin/npx</code> .
nothing	Node is not installed. <code>brew install node</code>

NVM USERS, READ THIS

nvm is a shell function, not a binary. It does not exist at all outside a terminal, so any nvm path in your config points at a specific node version that will break the day you upgrade. If you are handing this setup to learners, installing node via Homebrew is far more durable.

Rosetta confusion

If node was installed under Rosetta, or carried over by Migration Assistant from an Intel Mac, you can end up with an `x86_64` node in `/usr/local` on an Apple Silicon machine — sometimes alongside an ARM one in `/opt/homebrew`. Check:

```
file $(which node)
which -a npx
```

If you see two, point the config at the `/opt/homebrew` one. A mismatched pair produces confusing intermittent failures.

Homebrew is not in `/usr/local` here

On Apple Silicon, Homebrew installs to `/opt/homebrew`. Guides written for Intel Macs say `/usr/local` and will not work. If you copied a config from an older tutorial, this is your bug.

Permissions and Gatekeeper

Not usually a factor — npx and node are not quarantined. If you genuinely suspect it: `xattr -d com.apple.quarantine $(which node)`. Full Disk Access is **not** required for this setup; do not grant it chasing this problem.

7 macOS — Intel

Everything in Section 6 applies. The only difference is where Homebrew puts things — but it causes its own distinct confusion.

The prefix

	Intel	Apple Silicon
Homebrew prefix	<code>/usr/local</code>	<code>/opt/homebrew</code>
npx	<code>/usr/local/bin/npx</code>	<code>/opt/homebrew/bin/npx</code>

THE REVERSE TRAP

Most current tutorials are written on Apple Silicon and say `/opt/homebrew`. On an Intel Mac that path does not exist, so the server fails with `ENOENT` and the config looks perfectly correct. If you are on Intel, assume every path you copy from the internet needs changing.

Migrated machines

An Intel Mac restored from a Time Machine backup of an Apple Silicon Mac can have a stale, non-functional `/opt/homebrew` directory. It exists, so the config looks right, but the binaries inside are the wrong architecture. Always confirm with `which npx` rather than trusting that a path exists.

Older macOS versions

On macOS 11 and 12, Homebrew may warn it is unsupported. Node itself still works. If `brew install node` fails outright, use the official installer from nodejs.org — it installs to `/usr/local/bin/node`, which matches the Intel config in Section 2 exactly.

Verify after any change

```
/usr/local/bin/npx --version
```

If this prints a version number, the path in your config is correct. If it says "no such file or directory", it is not — go back to `which npx`.

8 Windows 10 / 11

Why cmd /c is required

On Windows, `npx` is not an executable — it is `npx.cmd`, a batch script. Claude Desktop cannot launch a batch file directly, so you route it through the command interpreter. This is why the macOS config shape does not work on Windows even with a corrected path.

Symptom

```
Error: spawn npx ENOENT
```

Fix

Use `"command": "cmd"` with `"/c"` as the first argument, exactly as shown in Section 2.

BACKSLASHES MUST BE DOUBLED IN JSON

If you ever put a Windows path in this file, every backslash must be escaped:

`"C:\\Program Files\\nodejs\\npx.cmd"` — correct

`"C:\Program Files\nodejs\npx.cmd"` — invalid JSON, breaks the whole file

Forward slashes also work and avoid the problem: `"C:/Program Files/nodejs/npx.cmd"`

If cmd /c still fails

Node is not on the system `PATH`. Check in Command Prompt:

```
where npx
```

If it prints...	What to do
<code>C:\Program Files\nodejs\npx.cmd</code>	Standard install. <code>cmd /c npx</code> should work — if not, restart your PC so <code>PATH</code> refreshes.
<code>%APPDATA%\nvm\...</code>	nvm-windows. Run <code>nvm use <version></code> once, then restart Claude.
"could not find files"	Node is not installed or not on <code>PATH</code> . Reinstall from nodejs.org and tick "Add to <code>PATH</code> ".

Quitting Claude completely

THE WINDOWS-SPECIFIC RESTART TRAP

Closing the Claude window on Windows leaves it running in the system tray. Your config change will not load, and it looks like the config is wrong when it is fine.

Right-click the Claude icon in the tray (bottom-right, possibly hidden under the `^` arrow) → **Quit**. If you cannot find it: `Ctrl+Shift+Esc` → Task Manager → end every `Claude.exe`.

Antivirus and SmartScreen

Third-party antivirus sometimes blocks `npx` downloading packages, or kills the long-lived connection. Symptom: Step 1 in Section 4 hangs with no output, or exits with a network error. Test by temporarily disabling real-time protection; if that fixes it, add an exclusion for your node install folder rather than leaving protection off.

Editing the file safely

Open `%APPDATA%\Claude\` by pasting it into the Explorer address bar. Edit with VS Code or Notepad++, not Notepad — Notepad can save as `claude_desktop_config.json.txt` without telling you, and Claude will never find it. In Explorer, turn on **View** → **File name extensions** so you can see what the file is actually called.

WSL

Claude Desktop is a Windows application and cannot launch a command inside WSL. Node must be installed on Windows itself. A WSL-only node install is invisible to Claude no matter what path you write.

9 Reading the logs

The logs answer the one question that matters: how far down the chain did it get before dying?

Where they are

Platform	Path
macOS	~/Library/Logs/Claude/mcp-server-gmail-assistant.log
Windows	%APPDATA%\Claude\logs\mcp-server-gmail-assistant.log

The filename follows your server's key in the config. There is also a combined `mcp.log` in the same folder.

```
# macOS – watch live while restarting Claude
tail -f ~/Library/Logs/Claude/mcp-server-gmail-assistant.log

# Windows PowerShell
Get-Content "$env:APPDATA\Claude\logs\mcp-server-gmail-assistant.log" -Wait -Tail 40
```

What the lines mean

Line	Meaning
Initializing server...	Claude read your config. If absent, it did not — check JSON and file location.
Using MCP server command: /opt/.../npx	The exact binary being launched. Compare against <code>which npx</code> .
Server started and connected successfully	Process launched and handshake completed.
[supergateway] Stdio server listening	The bridge is up. Chain is working.
spawn ... ENOENT	Wrong path. Sections 6–8.
SSE error: undefined	Idle reconnect. Harmless if tools work — see Section 5.
404	Workflow inactive, or Test URL used.

READING IT IN ORDER

Work top to bottom and find the last line that succeeded. Everything before it is healthy; the failure is immediately after. That alone usually identifies which of the four components is at fault without any further guessing.

10 Error → fix quick reference

Symptom	Where	Fix
Server not listed at all	All	Invalid JSON, wrong folder, or a duplicate <code>mcpServers</code> key. Validate with <code>json.tool</code> .
<code>spawn npx ENOENT</code>	macOS	Use the absolute path from <code>which npx</code> and keep the <code>env.PATH</code> block.
<code>spawn npx ENOENT</code>	Windows	Use <code>"command": "cmd"</code> with <code>"/c"</code> as first arg.
Path exists but still ENOENT	macOS	Wrong Homebrew prefix for your chip — <code>/opt/homebrew</code> (ARM) vs <code>/usr/local</code> (Intel).
Config edits do nothing	All	Not fully quit. ⌘Q on macOS; tray → Quit on Windows.
404 from n8n	All	Activate the workflow; use the Production URL.
Worked yesterday, dead today	All	You are on the Test URL, or supergateway auto-updated. Pin the version.
<code>expected string, received undefined</code>	n8n	n8n requires every AI field. Supply all params, or delete unused fields from the node.
Tools listed, calls return 403	Gmail	Credential lacks send/compose scope. Re-authorize from scratch.
<code>SSE error: undefined</code>	All	Ignore if tools work. Normal idle reconnect.
Other servers vanished	All	You pasted a second <code>mcpServers</code> block. Merge into the existing one.
Saved file, Claude can't see it	Windows	Notepad saved it as <code>.json.txt</code> . Enable file extensions in Explorer.
Step 1 hangs, no output	All	Proxy, VPN or antivirus blocking npm or SSE. Test on a hotspot.
Node only in WSL	Windows	Install node on Windows. Claude cannot reach into WSL.

11 Security before you go live

READ THIS BEFORE POINTING IT AT A REAL INBOX

With the trigger's Authentication set to `None`, the endpoint URL **is** the credential. The Gmail OAuth token lives inside n8n, so anyone who obtains that URL can read and send mail as you — no password, no second factor, nothing else needed.

An unauthenticated n8n Cloud URL with a guessable path like `/mcp/gmail-assistant` is a meaningful exposure, not a theoretical one.

For teaching and experimenting

- Use a throwaway Google account, never a personal or work inbox.
- Give the Gmail credential read-only scope until sending is actually being demonstrated.
- Deactivate the workflow when the session ends — that alone closes the endpoint.

For anything real

1. Set the MCP Server Trigger's **Authentication** to `Bearer` and generate a long random token.
2. Pass it through supergateway by adding a header argument after the URL:

```
{
  "args": [
    "-y",
    "supergateway",
    "--sse",
    "https://YOUR-INSTANCE.app.n8n.cloud/mcp/YOUR-PATH",
    "--header",
    "Authorization: Bearer YOUR_TOKEN_HERE"
  ]
}
```

Confirm the flag spelling for your version with `npm run supergateway --help` — it has changed between releases.

Also worth doing

- Use a long random path in the trigger node, not `gmail-assistant`. Obscurity is not security, but it stops casual discovery.
- Expose only the operations you need. If a workflow never needs to send, do not attach a Send node — the safest tool is the one that is not there.
- Treat the config file as a secret. It may contain bearer tokens and API keys; do not commit it, screenshot it, or paste it into a chat.

ONE HABIT WORTH FORMING

Anything pasted into a shared channel while debugging — a config file, a log, a screenshot of the n8n editor — can carry a live credential. Redact the URL and any token before sharing, and rotate anything that does get exposed.